

Java Interview Questions & Answers

20 questions spanning basic syntax to core OOP to advanced concepts

Basics

1. What is the difference between JDK, JRE, and JVM?

JVM (Java Virtual Machine) executes Java bytecode and provides platform independence. JRE (Java Runtime Environment) includes the JVM plus the standard libraries needed to run Java applications. JDK (Java Development Kit) includes the JRE plus development tools like the compiler (javac) and debugger, needed to write and build Java programs.

2. What is the difference between '==' and '.equals()' when comparing objects?

'==' compares references, checking whether two variables point to the exact same object in memory. '.equals()' compares logical/content equality, and its behavior depends on whether the class has overridden it — for example, String overrides equals() to compare character content rather than reference identity.

3. What is the difference between primitive types and wrapper classes?

Primitives (int, boolean, double, etc.) store raw values directly and aren't objects, offering better performance and lower memory overhead. Wrapper classes (Integer, Boolean, Double, etc.) wrap a primitive value in an object, enabling use in collections (which require objects), null values, and utility methods; autoboxing/unboxing lets Java convert between the two automatically.

4. What is the difference between 'final', 'finally', and 'finalize'?

'final' is a keyword used to make a variable constant, prevent method overriding, or prevent class inheritance. 'finally' is a block that always executes after a try/catch, regardless of whether an exception occurred, typically used for cleanup. 'finalize()' is a method called by the garbage collector before an object is destroyed, though its use is now discouraged in favor of try-with-resources.

5. What is the difference between an array and an ArrayList?

An array has a fixed size determined at creation and can hold primitives or objects. An ArrayList is part of the Collections framework, grows dynamically as elements are added, only holds objects (autoboxing wraps primitives), and provides built-in methods for adding, removing, and searching elements.

OOP Concepts

1. What are the four main pillars of object-oriented programming?

Encapsulation (bundling data and methods together while restricting direct access via access modifiers), inheritance (a class acquiring properties and behavior from a parent class), polymorphism (the ability of a method or object to take multiple forms, via overriding or overloading), and abstraction (hiding implementation details and exposing only essential features, via abstract classes or interfaces).

2. What is the difference between method overloading and method overriding?

Overloading means defining multiple methods with the same name but different parameter lists within the same class, resolved at compile time. Overriding means a subclass provides its own implementation

of a method already defined in its superclass, with the same signature, resolved at runtime based on the actual object type.

3. What is the difference between an abstract class and an interface?

An abstract class can have both abstract and concrete methods, constructors, and instance fields, but a class can extend only one abstract class. An interface (traditionally) declares only method signatures (though modern Java allows default and static methods), has no constructors, and a class can implement multiple interfaces — making interfaces the mechanism for achieving multiple inheritance of type in Java.

4. What is constructor overloading, and why would you use it?

Constructor overloading means defining multiple constructors in a class with different parameter lists, letting objects be created in different ways depending on which data is available at instantiation time — for example, a Book class with one constructor taking just a title, and another taking title, author, and price.

5. What is the 'super' keyword used for in Java?

'super' refers to the immediate parent class. It's used to call the parent class's constructor (super(...)), to invoke an overridden method's parent implementation (super.methodName()), or to access a parent class field that's been shadowed by a subclass field of the same name.

Collections & Exceptions

1. What is the difference between List, Set, and Map in the Collections framework?

A List (e.g. ArrayList, LinkedList) is an ordered collection that allows duplicate elements and index-based access. A Set (e.g. HashSet, TreeSet) does not allow duplicates and generally does not guarantee order. A Map (e.g. HashMap, TreeMap) stores key-value pairs, where each key is unique and maps to exactly one value.

2. What is the difference between HashMap and TreeMap?

HashMap stores key-value pairs with no guaranteed ordering, offering average $O(1)$ time for get/put operations via hashing. TreeMap stores entries in sorted order based on the keys' natural ordering (or a supplied Comparator), backed by a red-black tree, giving $O(\log n)$ operations but ordered iteration.

3. What is the difference between checked and unchecked exceptions?

Checked exceptions (e.g. IOException) are checked at compile time — a method must either handle them with try/catch or declare them with 'throws'. Unchecked exceptions (e.g. NullPointerException, ArithmeticException) extend RuntimeException and aren't checked at compile time, typically representing programming errors rather than recoverable conditions.

4. What is the try-with-resources statement, and what problem does it solve?

try-with-resources automatically closes any resource (like a file stream) that implements the AutoCloseable interface once the try block finishes, even if an exception occurs, eliminating the need for an explicit finally block to close resources manually and reducing the risk of resource leaks.

5. What is the difference between 'throw' and 'throws'?

'throw' is used inside a method body to actually raise/trigger a specific exception instance at that point in execution. 'throws' is used in a method's signature to declare that the method might propagate one or more checked exceptions, requiring callers to handle or further declare them.

Advanced Concepts

1. What is the Java Memory Model, and what's the difference between stack and heap memory?

The stack stores method call frames, local variables, and object references, following LIFO order and being automatically reclaimed when a method returns — it's fast but limited in size. The heap stores actual objects and is shared across all threads, managed by the garbage collector, and is where memory leaks and OutOfMemoryErrors typically originate.

2. What is garbage collection in Java, and can you name a common garbage collection algorithm?

Garbage collection automatically reclaims memory occupied by objects that are no longer reachable from any live reference, freeing developers from manual memory management. Common algorithms include Mark-and-Sweep (marking reachable objects, then sweeping away the rest) and Generational GC (dividing the heap into young and old generations, since most objects die young, which improves collection efficiency).

3. What is multithreading, and what is the difference between extending Thread and implementing Runnable?

Multithreading allows a program to execute multiple threads concurrently, sharing CPU time. Extending Thread means the class itself becomes a thread but can't extend any other class (Java has no multiple inheritance). Implementing Runnable is generally preferred, since the class only defines the task to run and can still extend another class, while the actual thread is created separately by passing the Runnable to a Thread object.

4. What is the 'volatile' keyword, and what problem does it solve?

'volatile' ensures that reads and writes to a variable go directly to and from main memory rather than a thread's local CPU cache, guaranteeing visibility of the latest value across threads. It solves visibility issues in multithreaded code but does not provide atomicity for compound operations like increment (count++).

5. What are Java Streams, and how do they differ from Collections?

A Collection is a data structure that stores elements in memory, while a Stream is a sequence of elements supporting functional-style operations (map, filter, reduce) computed lazily and, in some cases, in parallel. Streams don't store data themselves and don't modify the underlying source — they describe a pipeline of computation that produces a result or a new collection.